

Auditoría Técnica y Documentación Celda por Celda

Notebook: `CVM_Simulation_-_Case` — **Recomendador de Ofertas Cross-Sell (CellMov)**

Rol del auditor: Senior AI/ML Consultant & Tech Lead **Alcance:** 38 celdas (12 Markdown, 26 de código), incluyendo outputs de ejecución **Fecha de auditoría:** Julio 2026

0. Resumen Ejecutivo de la Auditoría

El notebook implementa un caso completo de **Customer Value Management (CVM)** para un operador de telecomunicaciones (CellMov): partiendo de una base de 100.000 leads sintéticos, ejecuta dos experimentos de referencia (oferta aleatoria y oferta fija de bajo costo), modela elasticidad-precio con regresión logística, y culmina con la construcción de una **política de recomendación tipo Contextual Bandit** con estrategia **epsilon-greedy decreciente** entrenada de forma secuencial sobre 10 lotes de 50.000 leads (simulando campañas quincenales).

Resultado final en el set oficial de validación (50.000 leads):

Estrategia	Conversión	Margen Total	Margen / Cliente
Random Offer (baseline)	0,9% (470)	\$18.829	\$0,38
Low-Cost Offer (baseline)	2,3% (1.143)	\$28.644	\$0,57
Contextual Bandit (solución)	5,7% (2.863)	\$133.848	\$2,68

Lift financiero: **7,11× vs. Random (+\$115.020)** y **4,67× vs. Low-Cost (+\$105.205)**.

La arquitectura del pipeline es: `EDA → Experimentos baseline → Modelado de elasticidad → Definición del espacio de acciones válidas → Bucle de aprendizaje bandit (10 batches, ε-greedy) → Evaluación en test interno → Validación oficial → Explicabilidad e impacto financiero`.

1. Documentación Celda por Celda

Celda 0 — [MARKDOWN | Título]

Contenido: Título del notebook: "*CVM Simulation - Case*". **Impacto en el pipeline:** Ninguno funcional; delimita el documento.

Celda 1 — [MARKDOWN | Advertencia metodológica]

Contenido: Advertencia crítica del autor del caso: el uso de IA está permitido de forma **colaborativa** (para expandir conocimiento y capacidades analíticas), pero resolver el caso de forma autónoma sin razonamiento paso a paso es detectable y descalificante. **Impacto en el pipeline:** Condiciona la metodología de trabajo: cada bloque de código posterior está acompañado de justificación de negocio explícita en comentarios, lo cual es coherente con esta restricción.

Celda 2 — [MARKDOWN | Contexto de negocio]

Contenido: Planteamiento del caso. CellMov es un operador convergente latinoamericano, líder en móvil pero segundo distante en banda ancha fija. Hallazgos clave que fundamentan el diseño posterior:

1. **La primera oferta presentada determina la conversión** (si no es atractiva, el cliente cuelga) → esto justifica el enfoque de *bandit* de un solo intento (one-shot decision), no un problema secuencial multi-turno por cliente.
2. **La propensión a convertir es elástica al precio** → justifica el modelado explícito de elasticidad (Celda 21).
3. El proceso actual asigna ofertas de forma **cuasi-aleatoria** → esto legitima que el experimento aleatorio sea un baseline realista y una fuente de datos de exploración no sesgada.

Impacto en el pipeline: Define la función objetivo del proyecto: **maximizar margen total** (no conversión), con personalización de la primera oferta.

Celda 3 — [MARKDOWN | Catálogo de productos]

Contenido: Portafolio ofertable con sus reglas de negocio (restricciones duras):

- **Internet:** Low Speed (150 Mbps) o High Speed (1.000 Mbps). **Obligatorio y excluyente** (exactamente uno).
- **TV:** Basic o Premium. **Opcional y excluyente** (ninguno, o uno de los dos).
- **OTT bundles:** Disney y Paramount, add-ons independientes.

- **Voz (VoIP):** add-on independiente.
- **Wall-to-Wall WiFi:** add-on premium independiente.

Impacto en el pipeline: Estas reglas se codifican **por construcción** en la Celda 28 (`build_action_space`), lo que garantiza que la función `sim()` nunca reciba combinaciones inválidas (sin internet, o con ambas TVs). Es una decisión de diseño superior al filtrado expost.

Celda 4 — [MARKDOWN | Tabla económica: precios y márgenes]

Contenido: Precio, COGS, depreciación CPE/instalación, amortización de comisión y margen por componente:

Producto	Precio	Margen	% Margen
Internet High Speed	\$40,00	\$31,61	79%
Internet Low Speed	\$25,00	\$18,83	75%
TV Basic	\$10,00	\$4,50	45%
TV Premium	\$23,00	\$11,78	51%
Disney Bundle	\$10,00	\$5,00	50%
Paramount Bundle	\$8,00	\$4,00	50%
VoIP Line	\$8,00	\$5,56	69%
Wall-to-Wall WiFi	\$4,00	\$2,25	56%

Impacto en el pipeline: Estos valores se hardcodean en los diccionarios `PRODUCT_MARGINS` y `PRODUCT_PRICES` de la Celda 28. Son la pieza que transforma el problema de **clasificación** (¿convertirá?) en un problema de **decisión económica** (margen esperado = $P(\text{conversión} \mid \text{cliente, oferta}) \times \text{margen de la oferta}$). Nótese la asimetría clave: High Speed Internet aporta $\sim 1,7\times$ el margen de Low Speed, y WiFi es el add-on de menor margen absoluto pero también de menor precio (\$4), lo que el bandit terminará explotando.

Celda 5 — [MARKDOWN | Diccionario de features de usuario]

Contenido: Variables disponibles por lead: `phone_number` (identificador), `age`, `socio_economic_segment` (A=muy afluente ... D=menor ingreso), `mobility_profile` (4

perfiles: solo hogar / hogar-trabajo / hogar-trabajo-tercer sitio / alta movilidad), `device`, y 8 variables de consumo de datos en GB (total, YouTube, WhatsApp, Netflix, Disney, otros OTT, portales de noticias, portales deportivos).

Impacto en el pipeline: Define el **espacio de contexto** del bandit. Las variables de consumo son proxies de comportamiento (afinidad streaming → TV Premium/OTT; permanencia en hogar → WiFi de pared a pared), y las categóricas segmentan la capacidad de pago.

Celda 6 — [MARKDOWN | Diseño experimental]

Contenido: Se anuncian dos experimentos sobre 100k usuarios: (1) **oferta aleatoria** a 50k usuarios; (2) **oferta fija de bajo costo** (Low Speed + Basic TV) a los otros 50k. **Impacto en el pipeline:** El experimento aleatorio cumple una función estadística crucial: al ser una asignación aleatoria de tratamiento, produce datos **sin sesgo de confusión (confounding)** entre características del cliente y oferta recibida, condición necesaria para estimar elasticidad-precio de forma causalmente válida en la Celda 21.

Celda 7 — [CÓDIGO | Preparación de datos — infraestructura del caso]

Código fuente:

```
python

#### Do not modify this cell, check how functions are called
from data_and_supporting_code.base import generate_lead_base
```

Explicación técnica: Celda inmutable ("Do not modify"). Importa la función generadora de leads sintéticos desde el paquete de soporte del caso. `generate_lead_base` es un generador estocástico de la población de clientes (equivalente al pipeline de leads del equipo de analytics). **Impacto en el pipeline:** Es la única fuente de datos de contexto para todo el notebook (bases experimentales, batches de entrenamiento y test interno).

Celda 8 — [CÓDIGO | Preparación de datos — creación de la base]

Código fuente:

```
python

lead_base = generate_lead_base(n=100_000)
lead_base.head()
```

Explicación técnica: Genera 100.000 leads y muestra las primeras 5 filas mediante `DataFrame.head()`. El output confirma 13 columnas con los tipos esperados: identificador numérico, edad entera, categóricas de segmento/movilidad/dispositivo y 8 numéricas continuas de consumo. **Impacto en el pipeline:** `lead_base` alimenta directamente el EDA (Celdas 10-12) y el split experimental (Celda 15).

Celda 9 — [MARKDOWN | Instrucción de EDA]

Instrucción del caso: ejecutar analítica descriptiva sobre la distribución de las variables base (Python o SQLite).

Celda 10 — [CÓDIGO | EDA — configuración y taxonomía de variables]

Código fuente:

```
python

# =====
# Descriptive Analytics: Lead Base Profiling
# =====

import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
PALETTE = "viridis"
plt.rcParams["figure.dpi"] = 100

# Explicit, business-meaningful ordering for categorical variables
segment_order = ["A", "B", "C", "D"]
mobility_order = [
    "Home - Mostly in one cell",
    "Home - Work profile",
    "Home - Work - Third Site Profile",
    "High mobility - Multiple Sites",
]

app_usage_cols = ["youtube_gb", "whatsapp_gb", "netflix_gb", "disney_gb", "othe
ott_cols = ["netflix_gb", "disney_gb", "other_ott_gb"]
numeric_consumption_cols = ["age", "total_usage_gb"] + app_usage_cols + ["news_
```

Explicación técnica:

- Importa `matplotlib.pyplot` y `seaborn`; fija estilo `whitegrid`, paleta `viridis` (perceptualmente uniforme y apta para daltonismo) y `figure.dpi = 100`.
- Define **órdenes explícitos con significado de negocio** para las categóricas: `segment_order = ["A", "B", "C", "D"]` (de mayor a menor ingreso) y `mobility_order` (de menor a mayor movilidad). Esto evita que seaborn ordene alfabéticamente y garantiza lecturas monotónicas en los gráficos.
- Agrupa las columnas en listas reutilizables: `app_usage_cols` (5 apps), `ott_cols` (subconjunto de streaming: Netflix, Disney, otros OTT) y `numeric_consumption_cols` (todas las numéricas + edad).

Impacto en el pipeline: No transforma datos; establece constantes de presentación y taxonomías que consumen las Celdas 11-12. La separación `ott_cols` anticipa la hipótesis comercial "consumo streaming → propensión a TV Premium / bundles OTT".

Celda 11 — [CÓDIGO | EDA — análisis univariado]

Código fuente:

```
python
```

```

# -----
# Section 1: Univariate Analysis – Who is our customer base?
# -----

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.countplot(
    data=lead_base, x="socio_economic_segment", order=segment_order,
    hue="socio_economic_segment", palette=PALETTE, legend=False, ax=axes[0],
)
axes[0].set_title("Lead Distribution by Socio-Economic Segment")
axes[0].set_xlabel("Socio-Economic Segment")
axes[0].set_ylabel("Number of Leads")

sns.countplot(
    data=lead_base, y="mobility_profile", order=mobility_order,
    hue="mobility_profile", palette=PALETTE, legend=False, ax=axes[1],
)
axes[1].set_title("Lead Distribution by Mobility Profile")
axes[1].set_xlabel("Number of Leads")
axes[1].set_ylabel("Mobility Profile")

plt.tight_layout()
plt.show()

# Age distribution
plt.figure(figsize=(8, 5))
sns.histplot(data=lead_base, x="age", bins=30, kde=True, color=sns.color_palette("magma"))
plt.title("Age Distribution of Leads")
plt.xlabel("Age (years)")
plt.ylabel("Number of Leads")
plt.tight_layout()
plt.show()

# Total data usage: histogram + boxplot side by side (distribution shape + outliers)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(data=lead_base, x="total_usage_gb", bins=40, kde=True, ax=axes[0],
             color=sns.color_palette("magma"))
axes[0].set_title("Total Data Usage Distribution")
axes[0].set_xlabel("Total Usage (GB)")
axes[0].set_ylabel("Number of Leads")

sns.boxplot(data=lead_base, y="total_usage_gb", ax=axes[1], color=sns.color_palette("magma"))

```

```
axes[1].set_title("Total Data Usage – Outlier Check")
axes[1].set_ylabel("Total Usage (GB)")

plt.tight_layout()
plt.show()
```

Explicación técnica:

- `sns.countplot` × 2 (subplots 1×2 vía `plt.subplots`): distribución de leads por segmento socioeconómico y por perfil de movilidad, usando los órdenes definidos. El patrón `hue=` + `legend=False` es el idioma moderno de seaborn ≥0.13 para colorear barras por categoría sin advertencias de deprecación.
- `sns.histplot(..., bins=30, kde=True)` para edad: histograma con estimación de densidad kernel superpuesta, que permite evaluar asimetría y modas.
- Para `total_usage_gb`: histograma (40 bins + KDE) y `sns.boxplot` lado a lado. La combinación es metodológicamente correcta: el histograma revela la forma de la distribución (típicamente sesgada a la derecha en consumo de datos) y el boxplot expone outliers vía la regla de 1,5×IQR.

Impacto en el pipeline: Diagnóstico de calidad y forma de los datos. Justifica implícitamente la posterior estandarización (`StandardScaler`) de las numéricas en el modelo (Celda 27): las magnitudes de consumo (0–30 GB) y edad (18–60+) están en escalas distintas y con sesgo.

Celda 12 — [CÓDIGO | EDA — análisis multivariado orientado a negocio]

Código fuente:

```
python
```



```

# -----
# Section 2: Multivariate Analysis – Business-driven insights
# -----

def boxplot_by_category(data, category_col, order, value_cols, title, xlabel):
    """Plot side-by-side boxplots of `value_cols` grouped by a categorical column"""
    melted = data.melt(
        id_vars=category_col, value_vars=value_cols,
        var_name="app", value_name="usage_gb",
    )
    plt.figure(figsize=(10, 5))
    sns.boxplot(
        data=melted, x=category_col, y="usage_gb", hue="app",
        order=order, palette=PALETTE, showfliers=False,
    )
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel("Usage (GB)")
    plt.legend(title="Application", bbox_to_anchor=(1.02, 1), loc="upper left")
    plt.tight_layout()
    plt.show()

# Streaming (OTT) usage vs. Socio-Economic Segment
# Why: identify which segments are more likely to buy OTT bundles / Premium TV.
boxplot_by_category(
    lead_base, "socio_economic_segment", segment_order, ott_cols,
    "Streaming (OTT) Usage by Socio-Economic Segment",
    "Socio-Economic Segment",
)

# Streaming (OTT) usage vs. Mobility Profile
# Why: home-based profiles may be better OTT/Premium-TV targets than high-mobility
boxplot_by_category(
    lead_base, "mobility_profile", mobility_order, ott_cols,
    "Streaming (OTT) Usage by Mobility Profile",
    "Mobility Profile",
)

# Mobility profile vs. total and app-level usage (mean comparison)
# Why: test whether high-mobility profiles (3 & 4) skew toward mobile data usage
# while home-based profiles (1 & 2) skew toward streaming – informs whether to
# Wall-to-Wall WiFi (home-based) vs. high mobile data plans (high-mobility).

```

```

mobility_summary = (
    lead_base.groupby("mobility_profile")[["total_usage_gb"] + app_usage_cols]
    .mean()
    .reindex(mobility_order)
)

mobility_summary.plot(kind="bar", figsize=(12, 6), colormap=PALETTE, width=0.8)
plt.title("Average Data Usage by Mobility Profile")
plt.xlabel("Mobility Profile")
plt.ylabel("Average Usage (GB)")
plt.xticks(rotation=15, ha="right")
plt.legend(title="Usage Type", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

# Correlation heatmap of numerical consumption variables
# Why: discover which applications are consumed together (e.g., YouTube & Whats
# to design custom bundles.
plt.figure(figsize=(9, 7))
corr = lead_base[numeric_consumption_cols].corr()
sns.heatmap(
    corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
    square=True, linewidths=0.5, cbar_kws={"label": "Correlation"},
)
plt.title("Correlation Heatmap – Age & Consumption Variables")
plt.tight_layout()
plt.show()

```

Explicación técnica:

- Define la función utilitaria `boxplot_by_category`, que aplica `DataFrame.melt` (pivote long-format con `id_vars`/`value_vars`) para poder graficar múltiples apps como `hue` en un solo boxplot agrupado. `showfliers=False` suprime outliers para legibilidad comparativa de medianas/cuartiles.
- Ejecuta 4 análisis, cada uno con su hipótesis comentada en el código:
 - OTT vs. segmento socioeconómico** → identificar segmentos objetivo de bundles OTT / TV Premium.
 - OTT vs. perfil de movilidad** → hipótesis: perfiles hogareños son mejores targets de OTT/TV.
 - Consumo medio por perfil de movilidad** (`groupby().mean().reindex()` + bar plot): contrasta si alta movilidad ⇒ más datos móviles y hogar ⇒ más streaming, informando el targeting de Wall-to-Wall WiFi.

4. **Heatmap de correlaciones** (`DataFrame.corr()` de Pearson + `sns.heatmap` con `annot=True`, `cmap="coolwarm"`, `center=0`): detecta co-consumo de aplicaciones para diseñar bundles.

Impacto en el pipeline: Este EDA es la fundamentación empírica de la ingeniería de variables: valida que segmento, movilidad y consumo streaming portan señal diferencial, exactamente las variables que el modelo de recompensa del bandit terminará ponderando con mayor peso (véase Celda 35: `Segment A` y `Home - Mostly in one cell` aparecen entre los 5 coeficientes más influyentes).

Celda 13 — [MARKDOWN | Sección "Simulation"]

Separador de sección.

Celda 14 — [MARKDOWN | Instrucción de split]

Instrucción: dividir `lead_base` en `random_base` (primeros 50.000 registros) y `low_offer_base` (segundos 50.000), respetando la convención de nombres exacta.

Celda 15 — [CÓDIGO | Preparación de datos — split experimental]

Código fuente:

```
python

# First 50,000 records (from 0 to 49,999)
random_base = lead_base.iloc[:50_000].reset_index(drop=True)

# Second 50,000 records (from 50,000 to 99,999) using the exact naming convention
low_offer_base = lead_base.iloc[50_000:100_000].reset_index(drop=True)

print(f"random_base size: {random_base.shape}")
display(random_base.head())
print(f"low_offer_base size: {low_offer_base.shape}")
display(low_offer_base.head())

# Assign low_offer_base to low_cost_base to ensure compatibility with subsequent steps
low_cost_base = low_offer_base
```

Explicación técnica:

- `DataFrame.iloc` realiza un **corte posicional determinista** (no aleatorio), tal como exige la consigna. `reset_index(drop=True)` reindexa desde 0 para evitar desalineaciones posicionales en merges/asignaciones posteriores.
- Se imprime `shape` y `head()` de ambas bases como verificación (50.000×13 cada una).
- Se crea el alias `low_cost_base = low_offer_base` (misma referencia en memoria, no copia) para compatibilizar con la nomenclatura que usa la celda inmutable 17.

Observación de auditoría: Al ser un split posicional y no estratificado, la validez comparativa entre grupos descansa en que `generate_lead_base` produzca filas i.i.d. (intercambiables). Dado que la base es sintética y generada en un solo llamado, la asunción es razonable; en datos reales se recomendaría aleatorizar el split o verificar balance de covariables entre grupos.

Impacto en el pipeline: Ambas bases alimentan las funciones de asignación de oferta de la Celda 17.

Celda 16 — [MARKDOWN | Instrucción]

Asignar ofertas usando funciones de la librería de experimentos.

Celda 17 — [CÓDIGO | Preparación de datos — asignación de tratamientos]

Código fuente:

```
python

### Do not modify this cell

from data_and_supporting_code.experiments import random_offer as random_offer_fn

random_offer = random_offer_fn(random_base)
low_cost_offer = fixed_offer(low_cost_base)

print(f"random_offer size: {random_offer.shape}")
display(random_offer.head())
print(f"low_cost_offer size: {low_cost_offer.shape}")
display(low_cost_offer.head())
```

Explicación técnica: Celda inmutable. Importa dos asignadores de tratamiento:

- `random_offer_fn`: adjunta a cada lead una combinación de oferta **aleatoria válida**, materializada en 8 flags binarios (`offer_*`) + un campo textual (`offer_description`). El import con alias (`as random_offer_fn`) evita la colisión de nombres con el DataFrame resultante (`random_offer`) — un detalle sutil pero necesario, ya que la función y el resultado compartirían identificador.
- `fixed_offer`: asigna a todos los leads la misma oferta (`Low Speed Internet + Basic TV`).

El output confirma que ambos DataFrames pasan de 13 a **22 columnas** (13 originales + 8 flags + descripción).

Impacto en el pipeline: Produce los dos datasets de tratamiento que consumirá la simulación (Celda 18). El experimento aleatorio, además, será el insumo del modelo de elasticidad (Celda 21) precisamente por su asignación no sesgada.

Celda 18 — [CÓDIGO | Simulación — ejecución de los experimentos baseline]

Código fuente:

```
python

from data_and_supporting_code.sim import sim

random_results = sim(random_offer)
low_cost_results = sim(low_cost_offer)

display(random_results.head())

print(f"Random offer converted:           {random_results['converted'].sum():,} ({random_results['converted'].sum()/len(random_results):,}%)")
print(f"Random Margin:                     {random_results['total_margin'].sum():,} ({random_results['total_margin'].sum()/len(random_results):,}%)")
print()
print(f"Low cost offer converted:           {low_cost_results['converted'].sum():,} ({low_cost_results['converted'].sum()/len(low_cost_results):,}%)")
print(f"Low cost offer Margin:               {low_cost_results['total_margin'].sum():,} ({low_cost_results['total_margin'].sum()/len(low_cost_results):,}%)")
```

Explicación técnica: Importa y ejecuta (`sim()`), el **entorno estocástico caja negra** del caso (equivalente a lanzar la campaña real). El output agrega tres columnas de resultado: `total_price` (precio de la oferta), `total_margin` (margen realizado: 0 si no convierte, margen de la oferta si convierte) y `converted` (booleano). Se imprimen KPIs agregados con `sum()` y formato f-string.

Resultados observados:

- Random: **367 conversiones (0,7%)**, margen \$13.355,83.
- Low-cost: **1.252 conversiones (2,5%)**, margen \$31.375,12.

Impacto en el pipeline: Establece los dos **benchmarks financieros** contra los que se medirá el recomendador, y confirma empíricamente la hipótesis de elasticidad del caso: la oferta barata convierte $\sim 3,4\times$ más que la aleatoria (cuyo precio medio es \$22,52 mayor).

Celda 19 — [MARKDOWN | Instrucción]

Ejecutar analítica descriptiva sobre resultados, modelar elasticidad-precio con los datos del experimento aleatorio y extraer insights de preferencia.

Celda 20 — [CÓDIGO | EDA de resultados — comparativa de experimentos]

Código fuente:

```
python
```

```

import statsmodels.api as sm
import pandas as pd
import numpy as np
import matplotlib.ticker as mtick

sns.set_style("whitegrid")
PALETTE = "viridis"

random_results = random_results.loc[:, ~random_results.columns.duplicated()]
low_cost_results = low_cost_results.loc[:, ~low_cost_results.columns.duplicated()]

# =====
# Section 1: Comparative Descriptive Analytics
# =====

offer_feature_cols = [
    "offer_high_speed_internet", "offer_low_speed_internet", "offer_basic_tv",
    "offer_premium_tv", "offer_disney_ott", "offer_paramount_ott",
    "offer_voice_line", "offer_wall_to_wall_wifi",
]

# 1a. Overall conversion rate: random (varied offers) vs. low-cost (fixed offer)
conversion_comparison = pd.DataFrame({
    "Experiment": ["Random Offer", "Low-Cost Offer"],
    "Conversion Rate": [random_results["converted"].mean(), low_cost_results["converted"].mean()],
    "Avg. Total Margin ($)": [random_results["total_margin"].mean(), low_cost_results["total_margin"].mean()],
})
print("=== Overall Conversion Rate Comparison ===")
print(conversion_comparison.round(4))

def bundle_performance(results_df, top_n=10):
    """Conversion rate and average margin per unique offer bundle."""
    return (
        results_df.groupby("offer_description")
        .agg(
            customers=("converted", "size"),
            conversion_rate=("converted", "mean"),
            avg_margin_per_customer=("total_margin", "mean"),
        )
        .sort_values("conversion_rate", ascending=False)
        .head(top_n)
    )

```

```

)

print("\n=== Top Bundles by Conversion Rate: Random Offer ===")
print(bundle_performance(random_results).round(4))

print("\n=== Top Bundles by Conversion Rate: Low-Cost Offer ===")

print(bundle_performance(low_cost_results).round(4))

# 1c. Individual component conversion rate: acceptance rate when a feature IS o
# (meaningful only in the random experiment, since the low-cost experiment neve
# varies these components)
component_conversion = pd.DataFrame({
    "Offer Component": [c.replace("offer_", "").replace("_", " ").title() for c in
    "Conversion Rate When Offered": [
        random_results.loc[random_results[c] == 1, "converted"].mean() for c in
    ],
}).sort_values("Conversion Rate When Offered", ascending=False)
print("\n=== Conversion Rate by Individual Offer Component (Random Offer) ===")
print(component_conversion.round(4))

plt.figure(figsize=(9, 5))
sns.barplot(
    data=component_conversion, x="Conversion Rate When Offered", y="Offer Compo
    hue="Offer Component", palette=PALETTE, legend=False,
)
plt.title("Conversion Rate by Offer Component (Random Offer Experiment)")
plt.xlabel("Conversion Rate When Component Is Offered")
plt.ylabel("")
plt.tight_layout()
plt.show()

```

Explicación técnica:

- Importa `statsmodels.api`, `pandas`, `numpy` y `matplotlib.ticker`.
- **Saneamiento defensivo:** `df.loc[:, ~df.columns.duplicated()]` elimina la columna `offer_description` duplicada que `sim()` devuelve (visible en el output de la Celda 18, donde la columna aparece dos veces). Sin este paso, agrupaciones y selecciones posteriores fallarían o producirían resultados ambiguos. Buena práctica de robustez.
- Construye `conversion_comparison`: conversión media y margen medio por experimento (Random: 0,73% / \$0,267 por cliente; Low-Cost: 2,50% / \$0,628 por cliente).

- `bundle_performance()`: `groupby("offer_description").agg()` con **agregación nombrada** (named aggregation, sintaxis `alias=("col", "func")`) para computar clientes, tasa de conversión y margen medio por bundle único; ordena descendente por conversión y toma top-10.
- **Análisis por componente:** para cada flag de oferta, calcula la conversión condicional `P(converted | componente ofrecido = 1)` con máscaras booleanas — válido **solo** en el experimento aleatorio (en el fijo no hay variación de componentes, como el propio comentario del código aclara). Se grafica con `sns.barplot` horizontal.

Hallazgos relevantes del output: el bundle con mayor conversión del experimento aleatorio es `Low Speed Internet` solo (6,26%) — el más barato del catálogo — coherente con elasticidad negativa; le siguen bundles High Speed premium, sugiriendo heterogeneidad de preferencias (hay un segmento dispuesto a pagar por velocidad).

Impacto en el pipeline: Motiva y valida la especificación del modelo de elasticidad de la celda siguiente, y da la primera evidencia de que la personalización (no una única oferta) es la estrategia óptima.

Celda 21 — [CÓDIGO | Modelado estadístico — elasticidad-precio (Logit)]

Código fuente:

```
python
```

```

# =====
# Section 2: Price Elasticity Modeling (Random Offer Experiment)
# =====

model_flags = [
    "offer_high_speed_internet", "offer_basic_tv", "offer_premium_tv",
    "offer_disney_ott", "offer_paramount_ott", "offer_voice_line",
]

X = sm.add_constant(random_results[["total_price"] + model_flags])
y = random_results["converted"].astype(int)

assert np.linalg.matrix_rank(X.values) == X.shape[1], "Design matrix is rank-deficient"

price_mix_model = sm.Logit(y, X).fit(disp=False)
print("\n=== Price Elasticity Model: Logit(converted) ~ total_price + product mix")
print(price_mix_model.summary())

price_coef = price_mix_model.params["total_price"]
print(f"\nHolding product mix fixed, each additional $1 in price changes the "
      f"log-odds of conversion by {price_coef:.4f} (odds ratio: {np.exp(price_coef):.4f})")
if price_mix_model.pvalues["total_price"] > 0.05:
    print("Note: this effect is not statistically significant at the 5% level in this run – expected, since once product mix is held fixed, the only remaining price variation comes from the $4 Wall-to-Wall WiFi add-on, which is in a narrow range that limits the power to detect a pure price effect.")

# 2b. Estimated demand curve: predicted conversion probability across price tier
# holding the product mix fixed at its average observed composition
price_grid = np.linspace(random_results["total_price"].min(), random_results["total_price"].max(), 100)
avg_mix = random_results[model_flags].mean()

X_grid = pd.DataFrame({"total_price": price_grid})
for flag in model_flags:
    X_grid[flag] = avg_mix[flag]
X_grid.insert(0, "const", 1.0)
X_grid = X_grid[X.columns] # match fitted model's column order exactly

predicted_curve = pd.DataFrame({
    "total_price": price_grid,
    "predicted_conversion_rate": price_mix_model.predict(X_grid), # kept as raw log-odds
})

```

```
plt.figure(figsize=(9, 5))
sns.lineplot(
    data=predicted_curve, x="total_price", y="predicted_conversion_rate",
    color=sns.color_palette(PALETTE, 1)[0],
)
plt.title("Estimated Demand Curve – Predicted Conversion vs. Price\n(Product Mix)")
plt.xlabel("Total Offer Price ($)")
plt.ylabel("Predicted Conversion Probability")

plt.gca().yaxis.set_major_formatter(mtick.PercentFormatter(xmax=1, decimals=1))
plt.tight_layout()
plt.show()
```

Explicación técnica:

- Especificación: $\text{Logit}(\text{converted}) \sim \text{const} + \text{total_price} + 6 \text{ flags de producto}$. Los flags incluidos son `high_speed_internet`, `basic_tv`, `premium_tv`, `disney_ott`, `paramount_ott`, `voice_line`; se **excluyen deliberadamente** `low_speed_internet` (colineal perfecto con high-speed: uno es el complemento del otro, dado que internet es obligatorio y excluyente) y `wall_to_wall_wifi`. La exclusión del complemento evita la trampa de variables dummy; la de WiFi deja variación de precio residual identificable.
- `sm.add_constant` añade el intercepto; `assert np.linalg.matrix_rank(X) == X.shape[1]` verifica formalmente que la matriz de diseño tenga **rango completo** (no colinealidad perfecta) antes de estimar — control de calidad poco habitual y muy recomendable.
- `sm.Logit(y, X).fit(dis=False)`: estimación por **máxima verosimilitud (MLE)** de una regresión logística, con inferencia clásica (errores estándar, z-stats, p-values).
- El código incluso contempla explícitamente el escenario de no significancia del precio, explicando que al controlar por mix de producto la única variación de precio remanente proviene del add-on de WiFi (\$4) — un rango estrecho que limita el poder estadístico. En esta ejecución sí resultó significativo.

Resultados de la estimación (n=50.000, Pseudo R²=0,073, LLR p≈1,9e-64):

Variable	Coef.	p-value	Lectura
const	-1,9225	0,012	Log-odds base
total_price	-0,0580	0,028	Cada \$1 adicional multiplica los odds de conversión por 0,9437 (-5,63% por dólar)
offer_high_speed_internet	+0,8167	0,023	Preferencia por velocidad, condicional al precio
offer_premium_tv	+1,1252	0,070	Positivo, marginalmente significativo
offer_voice_line	-1,3141	<0,001	La línea de voz destruye conversión
offer_basic_tv / disney / paramount	n.s.	—	Sin efecto detectable

- Segunda parte: **curva de demanda estimada**. Genera una grilla de 50 precios (`np.linspace`) entre min y max observados), fija el mix de producto en su composición media observada (`avg_mix`), reordena las columnas para que coincidan exactamente con el modelo ajustado (`X_grid = X_grid[X.columns]`) — detalle crítico en statsmodels, que empareja por posición) y grafica (`model.predict()`) (probabilidad 0-1) vs. precio con formateo porcentual (`PercentFormatter`).

Impacto en el pipeline: Provee la **fundamentación causal** del recomendador: (a) el precio importa (elasticidad negativa), (b) las preferencias por componentes son heterogéneas y algunos componentes son netamente destructivos (voz). El bandit posterior redescubrirá estos patrones de forma independiente (Celda 35), lo que actúa como triangulación metodológica.

Celda 22 — [CÓDIGO | EDA de resultados — síntesis de insights]

Código fuente:

```
python
```

```

# =====
# Section 3: Experiment Insights – Random Baseline vs. Low-Cost Strategy
# =====

random_conv = random_results["converted"].mean()
low_cost_conv = low_cost_results["converted"].mean()
random_price = random_results["total_price"].mean()
low_cost_price = low_cost_results["total_price"].mean()

print("\n=== Interpreting the Price Coefficient ===")
print(f"- Coefficient sign: {'negative' if price_coef < 0 else 'positive'} "
      f"({price_coef:.4f} log-odds per $1) – higher price "
      f"{'reduces' if price_coef < 0 else 'increases'} the odds of conversion, "
      "holding product mix fixed, consistent with standard demand theory.")
print(f"- Odds ratio {np.exp(price_coef):.4f}: each extra $1 charged multiplies "
      f"of conversion by {np.exp(price_coef):.4f} "
      f"({(1 - np.exp(price_coef)) * 100:.2f}% relative change in odds per doll

print("\n=== Conversion Shift: Random Baseline -> Low-Cost Strategy ===")
print(f"- Random offer: avg. price ${random_price:.2f} -> conversion rate {ra
print(f"- Low-cost offer: avg. price ${low_cost_price:.2f} -> conversion rate {
print(f"- Lowering price by ${random_price - low_cost_price:.2f} on average shi
      f"({low_cost_conv - random_conv) * 100:.2f} percentage points "
      f"({low_cost_conv / random_conv:.2f}x baseline), directionally consistent
      "price coefficient estimated above – though the low-cost offer also strip
      "(Premium TV, OTT bundles, Voice, WiFi), so part of this shift reflects a
      "less confusing product mix in addition to the pure price effect.")

```

Explicación técnica: Celda puramente narrativa-cuantitativa: imprime la interpretación del coeficiente de precio (signo, odds ratio 0,9437, –5,63% de odds por dólar) y contrasta los dos experimentos: bajar el precio medio de \$59,52 a \$37,00 (–\$22,52) elevó la conversión de 0,73% a 2,50% (3,41× el baseline). El propio texto advierte, con rigor, que el shift **no es un efecto precio puro**: la oferta low-cost también simplifica el mix (elimina add-ons), por lo que parte del efecto es atribuible a un producto más simple. Esto es un reconocimiento correcto de **confusión entre precio y composición** en la comparación entre experimentos.

Impacto en el pipeline: Cierra la fase diagnóstica y justifica el salto a una política personalizada: ni "todo barato" ni "aleatorio" son óptimos; el óptimo depende del cliente.

2. Sección Crítica — Núcleo de Machine Learning (Celdas 23–30)

Celda 23 — [MARKDOWN | Sección "Building your recommender model"]

Separador de sección.

Celda 24 — [CÓDIGO | Preparación de datos — bases de entrenamiento y test]

Código fuente:

```
python

### Do not modify this cell

for i in range(1, 11):
    globals()[f"train_leads_optimized_{i}"] = generate_lead_base(n=50_000)

#This loop created ten train datasets (train_leads_optimized_1, train_leads_opt
display(train_leads_optimized_1.head())

test_leads_optimized = generate_lead_base(n=50_000)
```

Explicación técnica: Celda inmutable. Genera **10 lotes de entrenamiento de 50.000 leads** (inyectados en el espacio de nombres global vía `globals()`), simulando 10 corridas quincenales de campaña) y un lote de test interno de 50.000. Total: 500.000 leads de entrenamiento como **presupuesto máximo de interacción** con `sim()`. **Impacto en el pipeline:** Impone la restricción de **eficiencia muestral** central del caso: la política solo puede aprender de estas 500k interacciones (10 llamadas a `sim()` de 50k cada una), replicando el costo y la latencia de campañas reales.

Celda 25 — [MARKDOWN | Reglas del recomendador]

Contenido: Especificación contractual de la solución:

- Solo se pueden usar los 10 batches para entrenar; el test se hace con `test_leads_optimized`.
- El DataFrame de oferta debe contener las 13 columnas originales **sin transformar** + los 8 flags one-hot de oferta; `sim()` falla si falta alguna columna, si un cliente no tiene internet o si se ofrecen ambas TVs.
- `sim()` es **caja negra estocástica**: prohibido ingeniería inversa, asignación determinista basada en `sim()` o evaluación exhaustiva de combinaciones.

- Se exige una **política de recomendación basada en aprendizaje** (RL, contextual bandits o similar) que aprenda de interacciones limitadas, **balancee exploración y explotación** y **generalice** a clientes no vistos (no una lookup table). Criterios de evaluación: metodología, eficiencia muestral, generalización y claridad.

Impacto en el pipeline: Este contrato explica todas las decisiones de diseño de las Celdas 26-30 (y el cumplimiento se audita en la sección 3 de este documento).

Celda 26 — [CÓDIGO | Modelado — imports del bandit]

Código fuente:

```
python

# =====
# Contextual Bandit - Offer Personalization Policy
# =====

import itertools
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

sns.set_style("whitegrid")
PALETTE = "viridis"
```

Explicación técnica: Importa las piezas de scikit-learn del modelo de recompensa:

`ColumnTransformer` (preprocesamiento heterogéneo por tipo de columna), `OneHotEncoder`, `StandardScaler`, `Pipeline` (encadenamiento preprocesado→clasificador con fit/predict unificado, previniendo *data leakage* de estadísticas de escala) y `LogisticRegression`.

También `itertools` para la enumeración combinatoria del espacio de acciones. **Impacto en el pipeline:** Habilita las Celdas 27-29.

Celda 27 — [CÓDIGO | Ingeniería de variables — preprocesador contexto+acción]

Código fuente:

```
python
```

```
# =====
# SECTION 1: Feature Engineering
# =====

CATEGORICAL_COLS = ["socio_economic_segment", "mobility_profile", "device"]
NUMERIC_COLS = [
    "age", "total_usage_gb", "youtube_gb", "whatsapp_gb", "netflix_gb",
    "disney_gb", "other_ott_gb", "news_portals_gb", "sports_portals_gb",
]
OFFER_FLAG_COLS = [
    "offer_high_speed_internet", "offer_low_speed_internet", "offer_basic_tv",
    "offer_premium_tv", "offer_disney_ott", "offer_paramount_ott",
    "offer_voice_line", "offer_wall_to_wall_wifi",
]

# One shared preprocessing pipeline for context + action features:
# - categorical customer attributes -> one-hot (simple, fully interpretable)
# - numeric usage attributes -> standardized (mean 0, unit variance)
# - offer flags -> passthrough (already 0/1)

preprocessor = ColumnTransformer(transformers=[
    ("categorical", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL_COLS),
    ("numeric", StandardScaler(), NUMERIC_COLS),
    ("offer_flags", "passthrough", OFFER_FLAG_COLS),
])
```

Explicación técnica: Define la taxonomía de features y el preprocesador compartido:

- `CATEGORICAL_COLS` = segmento, movilidad, dispositivo → `OneHotEncoder(handle_unknown="ignore")`. El parámetro `handle_unknown="ignore"` es crítico en producción/test: si aparece una categoría no vista en entrenamiento, se codifica como vector de ceros en lugar de lanzar excepción, garantizando robustez del scoring sobre leads nuevos.
- `NUMERIC_COLS` = edad + 8 consumos → `StandardScaler()` (media 0, varianza 1).
Fundamentación: la regresión logística con solver por gradiente converge mejor y sus coeficientes son comparables entre sí cuando las features comparten escala; además, la regularización L2 implícita (ver tabla de hiperparámetros) penalizaría injustamente features de gran magnitud sin estandarizar.
- `OFFER_FLAG_COLS` = los 8 flags de oferta → `passthrough` (ya son binarios 0/1).

El `ColumnTransformer` aplica cada transformación a su bloque de columnas y concatena horizontalmente el resultado.

Decisión arquitectónica clave (modelado de la acción como feature): el modelo aprende una única función $P(\text{conversión} \mid \text{contexto}, \text{acción})$ donde la acción entra como 8 features binarias, en lugar de entrenar 96 modelos independientes (uno por brazo). Esto es el enfoque de **bandit con acciones parametrizadas / feature-based** y es lo que permite **generalizar entre acciones**: la información aprendida sobre, p. ej., el efecto de `offer_voice_line` en un bundle se transfiere a los 47 bundles restantes que lo contienen. Con 96 brazos y ~0,7%-5% de conversión, entrenar por brazo sería estadísticamente inviable con 500k muestras.

Impacto en el pipeline: Este preprocesador se re-ajusta en cada iteración del bucle de entrenamiento (Celda 29) dentro del `Pipeline`, y es el mismo objeto conceptual usado para el scoring masivo de candidatos en `select_best_actions`.

Celda 28 — [CÓDIGO | Modelado — espacio de acciones y política]

Código fuente:

```
python
```

```

# =====
# SECTION 2: Action Space & Constraint Mapping
# =====

PRODUCT_MARGINS = {
    "offer_high_speed_internet": 31.61, "offer_low_speed_internet": 18.83,
    "offer_basic_tv": 4.50, "offer_premium_tv": 11.78,
    "offer_disney_ott": 5.00, "offer_paramount_ott": 4.00,
    "offer_voice_line": 5.56, "offer_wall_to_wall_wifi": 2.25,
}

PRODUCT_PRICES = {
    "offer_high_speed_internet": 40, "offer_low_speed_internet": 25,
    "offer_basic_tv": 10, "offer_premium_tv": 23,
    "offer_disney_ott": 10, "offer_paramount_ott": 8,
    "offer_voice_line": 8, "offer_wall_to_wall_wifi": 4,
}

def build_action_space() -> pd.DataFrame:
    """
    Enumerate every VALID offer combination:
        - Internet is mandatory and exclusive: exactly one of
          {high_speed, low_speed} is 1 (2 options).
        - TV is optional and mutually exclusive: none / basic / premium
          (3 options – never both).
        - Disney, Paramount, Voice, Wall-to-Wall WiFi are independent add-ons
          (2^4 = 16 combinations).
    Total valid actions: 2 * 3 * 16 = 96. Invalid combinations (no internet,
    or both TV options) are never constructed, so hard constraints are
    satisfied by design rather than filtered after the fact.
    """
    internet_options = [
        {"offer_high_speed_internet": 1, "offer_low_speed_internet": 0},
        {"offer_high_speed_internet": 0, "offer_low_speed_internet": 1},
    ]
    tv_options = [
        {"offer_basic_tv": 0, "offer_premium_tv": 0},
        {"offer_basic_tv": 1, "offer_premium_tv": 0},
        {"offer_basic_tv": 0, "offer_premium_tv": 1},
    ]
    addon_flags = ["offer_disney_ott", "offer_paramount_ott", "offer_voice_line"]
    addon_combos = list(itertools.product([0, 1], repeat=len(addon_flags)))

```

```

actions = [
    {**internet, **tv, **dict(zip(addon_flags, combo))}
    for internet in internet_options
    for tv in tv_options
    for combo in addon_combos
]

```

```

action_table = pd.DataFrame(actions)[OFFER_FLAG_COLS].reset_index(drop=True)
action_table.insert(0, "action_id", action_table.index)
action_table["action_margin"] = sum(action_table[c] * m for c, m in PRODUCT_
action_table["action_price"] = sum(action_table[c] * p for c, p in PRODUCT_

```

```

# Defensive check: hard constraints must hold for every constructed action
assert (action_table[["offer_high_speed_internet", "offer_low_speed_interne
assert (action_table[["offer_basic_tv", "offer_premium_tv"]].sum(axis=1) <=
return action_table

```

```

ACTION_TABLE = build_action_space()
print(f"Valid action space size: {len(ACTION_TABLE)}")

```

```

def select_best_actions(leads_df: pd.DataFrame, action_table: pd.DataFrame, mod
"""
Pure-exploitation action selection. Cross-joins every lead with every
valid action, scores expected margin for each (customer, action) pair in
one vectorized pass against the fitted model, and returns the
highest-expected-margin action per lead (row-aligned with leads_df).
This never calls sim() – it only scores our own fast internal model.
"""
leads = leads_df.reset_index(drop=True).copy()
leads["_row_id"] = leads.index

candidates = leads.merge(action_table, how="cross")
X_candidates = candidates[CATEGORICAL_COLS + NUMERIC_COLS + OFFER_FLAG_COLS
candidates["predicted_conversion"] = model.predict_proba(X_candidates)[: , 1
candidates["expected_margin"] = candidates["predicted_conversion"] * candid

best_idx = candidates.groupby("_row_id")["expected_margin"].idxmax()
best = candidates.loc[best_idx].sort_values("_row_id")
return best[OFFER_FLAG_COLS].reset_index(drop=True)

```

```

def epsilon_greedy_actions(leads_df, action_table, model, epsilon: float, rng:

```

```

"""
Epsilon-greedy policy: with probability `epsilon`, assign a uniformly
random VALID action (exploration); otherwise assign the model's highest
expected-margin action (exploitation). With no trained model yet (cold
start), every customer explores – there is nothing to exploit.
"""
n = len(leads_df)
random_ids = rng.integers(0, len(action_table), size=n)
random_actions = action_table.iloc[random_ids][OFFER_FLAG_COLS].reset_index()

if model is None:
    return random_actions

greedy_actions = select_best_actions(leads_df, action_table, model)
explore_mask = rng.random(n) < epsilon

chosen = greedy_actions.copy()
chosen[explore_mask] = random_actions[explore_mask].values
return chosen.astype(int)

```

Explicación técnica: Tres bloques:

(a) **Economía de las acciones.** `PRODUCT_MARGINS` y `PRODUCT_PRICES` transcriben la tabla de la Celda 4.

(b) `build_action_space()` — enumeración estructurada del espacio de acciones válidas:

- Internet: 2 opciones mutuamente excluyentes (dicts explícitos).
- TV: 3 opciones (ninguna / basic / premium), nunca ambas.
- Add-ons: `itertools.product([0,1], repeat=4)` → 16 combinaciones.
- Total: $2 \times 3 \times 16 = 96$ **acciones válidas**, construidas por comprensión de dicts con `merge` (`{**internet, **tv, **addons}`). A cada acción se le precomputa `action_margin` y `action_price` como producto punto `flags × diccionarios`.
- Dos `assert` verifican formalmente las restricciones duras (exactamente un internet; a lo sumo una TV). **Las combinaciones inválidas jamás se construyen** — cumplimiento de restricciones *by design*, no por filtrado, eliminando toda posibilidad de que `sim()` falle por oferta ilegal.

(c) **Funciones de política:**

- `select_best_actions(leads, action_table, model)` — **explotación pura, vectorizada:**
 1. `leads.merge(action_table, how="cross")` genera el producto cartesiano cliente × acción (para 50k leads: 4,8 millones de filas candidatas).

2. `model.predict_proba(X)[: , 1]` puntúa la probabilidad de conversión de cada par en **una sola pasada vectorizada** (columna 1 = clase positiva).
 3. `expected_margin = P(conversión) × action_margin` — la función de valor económico.
 4. `groupby("_row_id")["expected_margin"].idxmax()` selecciona el argmax por cliente y devuelve los flags alineados por fila. El docstring subraya que esto **nunca llama a `sim()`**: solo puntúa el modelo interno (rápido y gratuito), respetando la prohibición de evaluación exhaustiva contra el entorno.
- `epsilon_greedy_actions(leads, action_table, model, epsilon, rng)` — **política de exploración/explotación**:
 - Con probabilidad ϵ por cliente (máscara `rng.random(n) < epsilon`), asigna una acción válida uniforme (`rng.integers(0, 96)`); con probabilidad $1-\epsilon$, la acción greedy del modelo.
 - **Cold start explícito**: si `model is None` (batch 1), todos exploran — "no hay nada que explotar".
 - Usa `np.random.Generator` (API moderna de NumPy) inyectado como parámetro, lo que hace la aleatoriedad **reproducible y testeable**.

Impacto en el pipeline: Define íntegramente el mecanismo de decisión.

`epsilon_greedy_actions` gobierna el entrenamiento (Celda 29); `select_best_actions` gobierna la inferencia final (Celda 30).

Celda 29 — [CÓDIGO | Modelado — bucle de entrenamiento secuencial (batches 1-10)]

Código fuente:

```
python
```

```

# =====
# SECTION 3: Sequential Training Loop (Batches 1-10)
# =====
# Epsilon decays from full exploration (batch 1, no model exists yet) down to
# mostly exploitation (batch 10), reflecting growing confidence in the model
# as more biweekly campaign data accumulates.

EPSILON_SCHEDULE = np.linspace(1.0, 0.05, 10)
RNG = np.random.default_rng(42)

bandit_pipeline = None    # no policy trained yet (cold start)
training_log = []         # accumulated (context, chosen action, reward) rows
batch_performance = []    # per-batch KPIs, to visualize the learning curve

for batch_idx in range(1, 11):
    leads_batch = globals()[f"train_leads_optimized_{batch_idx}"]
    epsilon = EPSILON_SCHEDULE[batch_idx - 1]

    # 1. Select an offer for each customer via the epsilon-greedy policy
    chosen_actions = epsilon_greedy_actions(leads_batch, ACTION_TABLE, bandit_p

    # 2. Assemble the offer dataframe: original columns untouched + offer flags
    offer_batch = leads_batch.copy()
    offer_batch[OFFER_FLAG_COLS] = chosen_actions.values

    # 3. Run the simulation – the ONLY source of ground-truth reward.
    results_batch = sim(offer_batch)

    # 4. Log this batch's (context, action, reward) rows for training
    batch_rows = leads_batch[CATEGORICAL_COLS + NUMERIC_COLS].reset_index(drop=
    batch_rows[OFFER_FLAG_COLS] = chosen_actions.values
    batch_rows["converted"] = results_batch["converted"].astype(int).values
    training_log.append(batch_rows)

    # 5. Retrain the policy on ALL data collected so far (cumulative retraining
    cumulative_data = pd.concat(training_log, ignore_index=True)
    X_train = cumulative_data[CATEGORICAL_COLS + NUMERIC_COLS + OFFER_FLAG_COLS
    y_train = cumulative_data["converted"]

    bandit_pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression(max_iter=1000, class_weight="balanced
    ])

```

```

bandit_pipeline.fit(X_train, y_train)

# 6. Track batch KPIs
batch_performance.append({
    "batch": batch_idx,
    "epsilon": epsilon,
    "conversion_rate": results_batch["converted"].mean(),
    "total_margin": results_batch["total_margin"].sum(),
    "avg_margin_per_customer": results_batch["total_margin"].mean(),
})
print(f"Batch {batch_idx:>2}: epsilon={epsilon:.2f} | "
      f"conversion={results_batch['converted'].mean():.2%} | "
      f"margin=${results_batch['total_margin'].sum():.0f}")

performance_df = pd.DataFrame(batch_performance)

# Learning curve: does the policy actually improve as epsilon decays?
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.lineplot(data=performance_df, x="batch", y="conversion_rate", marker="o",
             ax=axes[0], color=sns.color_palette(PALETTE, 1)[0])
axes[0].set_title("Bandit Learning Curve: Conversion Rate by Batch")
axes[0].set_xlabel("Training Batch (Biweekly Run)")
axes[0].set_ylabel("Conversion Rate")

sns.lineplot(data=performance_df, x="batch", y="total_margin", marker="o",
             ax=axes[1], color=sns.color_palette(PALETTE, 1)[0])
axes[1].set_title("Bandit Learning Curve: Total Margin by Batch")
axes[1].set_xlabel("Training Batch (Biweekly Run)")
axes[1].set_ylabel("Total Margin ($)")
plt.tight_layout()
plt.show()

```

Explicación técnica. Es el corazón del sistema: un bucle de **aprendizaje on-policy** por lotes que emula 10 campañas quincenales.

Por cada batch (i):

1. **Selección de acciones** con `epsilon_greedy_actions` (ϵ según el schedule; batch 1 = exploración total por cold start).
2. **Ensamblado del DataFrame de oferta:** copia del batch original (columnas intactas, como exige el contrato) + flags elegidos.
3. **Interacción con el entorno:** `results_batch = sim(offer_batch)` — la **única fuente de recompensa real**; exactamente una llamada por batch, 10 en total.

- 4. **Logging de la tupla (contexto, acción, recompensa):** features del cliente + flags + `converted` (0/1) se apilan en `training_log`.
- 5. **Reentrenamiento acumulativo:** `pd.concat(training_log)` une **todos** los datos históricos y se reajusta desde cero un `Pipeline(preprocessor → LogisticRegression)`. El reentrenamiento full-history (en lugar de incremental/partial_fit) es correcto aquí: garantiza que el modelo use toda la evidencia con pesos consistentes y evita el olvido catastrófico; con 500k filas y regresión logística el costo computacional es trivial.
- 6. **Tracking de KPIs** por batch (ϵ , conversión, margen total y medio) para la curva de aprendizaje, graficada al final con `sns.lineplot` (conversión y margen vs. batch).

Curva de aprendizaje observada (output real):

Batch	ϵ	Conversión	Margen
1	1,00	0,95%	\$18.075
2	0,89	1,19%	\$25.313
3	0,79	1,37%	\$30.539
4	0,68	1,85%	\$42.462
5	0,58	2,08%	\$49.452
6	0,47	2,26%	\$54.931
7	0,37	2,60%	\$63.586
8	0,26	1,75%	\$43.634
9	0,16	3,80%	\$95.142
10	0,05	5,40%	\$128.884

Tendencia monótona creciente en conversión y margen a medida que ϵ decae (con una caída puntual en el batch 8, atribuible a la varianza estocástica del entorno y/o a una actualización de política transitoriamente subóptima — comportamiento normal en bandits, no un defecto), culminando en 5,40% de conversión y \$128.884 por batch: ~7× el margen del batch inicial aleatorio. **El sistema demuestra aprendizaje efectivo dentro del presupuesto de interacción.**

Impacto en el pipeline: Produce el artefacto final `bandit_pipeline` (política entrenada con los 500k ejemplos acumulados), consumido por la Celda 30 y por la validación oficial (Celda 33).

Celda 30 — [CÓDIGO | Modelado — evaluación en test interno]

Código fuente:

```
python
```

```

# =====
# SECTION 4: Final Test Evaluation
# =====

def bandit_function(leads_df: pd.DataFrame) -> pd.DataFrame:
    """
    Pure-exploitation policy (epsilon=0): assign each customer the valid
    offer with the highest model-predicted expected margin, using the final
    policy trained across all 10 batches. This is the function used both for
    our own internal check below and for the notebook's official final
    validation gate later on.
    """
    best_actions = select_best_actions(leads_df, ACTION_TABLE, bandit_pipeline)
    offer_df = leads_df.copy()
    offer_df[OFFER_FLAG_COLS] = best_actions.values
    return offer_df

optimized_offer_check = bandit_function(test_leads_optimized)
optimized_results_check = sim(optimized_offer_check)

random_offer_check = random_offer_fn(test_leads_optimized)
random_results_check = sim(random_offer_check)
low_cost_offer_check = fixed_offer(test_leads_optimized)
low_cost_results_check = sim(low_cost_offer_check)

comparison_check = pd.DataFrame({
    "Strategy": ["Random Offer", "Low-Cost Offer", "Contextual Bandit"],
    "Conversion Rate": [
        random_results_check["converted"].mean(),
        low_cost_results_check["converted"].mean(),
        optimized_results_check["converted"].mean(),
    ],
    "Total Margin ($)": [
        random_results_check["total_margin"].sum(),
        low_cost_results_check["total_margin"].sum(),
        optimized_results_check["total_margin"].sum(),
    ],
})

print("\n=== Internal Test Evaluation ===")
print(comparison_check.round(4))

```

Explicación técnica:

- Define `bandit_function(leads_df)`: envoltorio de **inferencia en explotación pura** ($\epsilon=0$) — asigna a cada cliente la acción de máximo margen esperado según la política final, devolviendo el DataFrame con el formato contractual exacto. Es la interfaz única de despliegue: se usa tanto para el chequeo interno como para la validación oficial.
- Evalúa las tres estrategias **sobre el mismo conjunto** `test_leads_optimized` (50k leads jamás vistos en entrenamiento): bandit, random (`random_offer_fn`) y low-cost (`fixed_offer`), cada una pasada por `sim()`.

Resultado del test interno:

Estrategia	Conversión	Margen Total
Random	0,76%	\$13.528
Low-Cost	2,81%	\$35.259
Contextual Bandit	3,61%	\$88.437

Impacto en el pipeline: Primera evidencia de **generalización a clientes no vistos** (criterio explícito del caso): el bandit multiplica $\times 2,5$ el margen del mejor baseline en datos frescos. Sirve de *sanity check* antes de la validación oficial. Nótese que evaluar los tres brazos sobre la misma base elimina la varianza entre poblaciones, un diseño de comparación pareada correcto.

3. Foco en Machine Learning — Análisis Profundo del Modelo

3.1 Tipo de modelo: Contextual Bandit ϵ -greedy con modelo de recompensa lineal

Familia algorítmica. La solución es un **Contextual Multi-Armed Bandit** con:

- **Espacio de acciones estructurado:** 96 brazos válidos, parametrizados por 8 features binarias (no brazos atómicos independientes).
- **Modelo de recompensa (reward model):** Regresión Logística $P(\text{conversión} \mid \text{contexto}, \text{acción})$ — es decir, un bandit de la familia de "**greedy sobre modelo de valor lineal en features conjuntas**", conceptualmente emparentado con LinUCB/regresión disjunta pero con exploración por ϵ -greedy en lugar de bonos de incertidumbre (UCB) o muestreo de posterior (Thompson Sampling).
- **Función de valor de decisión:** $\text{margen esperado} = P(\text{conversión}) \times \text{margen de la acción}$ — no se maximiza conversión sino **valor económico**, alineando el objetivo del modelo con el objetivo de negocio (mandato de la CMO).

- **Estrategia de exploración:** ϵ -greedy con annealing lineal de 1,0 $\rightarrow$ 0,05 en 10 pasos.
- **Régimen de aprendizaje:** on-policy, por lotes, con reentrenamiento acumulativo (full-history refit).

¿Por qué es la familia correcta para este problema?

1. **El problema ES un bandit, no un MDP completo:** cada cliente recibe una única oferta y la recompensa es inmediata (convierte o cuelga, según el contexto de negocio de la Celda 2). No hay estado que evolucione ni recompensas diferidas $\rightarrow$ RL completo (Q-learning, policy gradients) sería sobre-ingeniería innecesaria y menos eficiente en muestras.
2. **Contextual (no MAB simple):** el EDA y el logit demostraron heterogeneidad de preferencias (segmento A vs. D, hogareños vs. móviles); una política sin contexto colapsaría en la mejor oferta media, dejando valor sobre la mesa.
3. **Eficiencia muestral:** con 96 acciones y conversión base $<1\%$, la parametrización lineal de la acción comparte información estadística entre brazos, aprendiendo de $\sim 500k$ muestras lo que un bandit tabular necesitaría millones para estimar.
4. **Cumplimiento del contrato del caso:** aprende de interacciones limitadas (10 llamadas a `sim`), balancea exploración/explotación explícitamente (ϵ -schedule) y generaliza vía modelo paramétrico (no lookup table).
5. **Interpretabilidad:** los coeficientes del modelo lineal son directamente legibles por marketing (explotado en la Celda 35) — ventaja decisiva frente a alternativas de caja negra (GBM/red neuronal como reward model) en un contexto de auditoría y adopción ejecutiva.

Propiedades inherentes y trade-offs asumidos:

- **Ventaja:** convexidad del problema de optimización (MLE logístico) $\rightarrow$ entrenamiento estable, reproducible, sin tuning de arquitectura.
- **Limitación:** linealidad en log-odds $\rightarrow$ no captura interacciones contexto $\times$ acción salvo las implícitas en las features (p. ej., no puede aprender nativamente "Disney OTT convierte más solo si disney_gb es alto"; solo efectos aditivos). Un GBM o términos de interacción explícitos capturarían más señal a costa de interpretabilidad y riesgo de sobreajuste con recompensa tan escasa.
- **Limitación de exploración:** ϵ -greedy explora uniformemente (no dirige la exploración hacia acciones inciertas como UCB/Thompson); es sub-óptima en regret teórico pero simple, robusta y suficiente dado el presupuesto de 500k muestras.

3.2 Diccionario de Hiperparámetros

Todos los parámetros configurados de forma explícita en el código, sin omisiones:

Hiperparámetro	Valor Asignado	Definición Técnica
<code>LogisticRegression.max_iter</code>	1000	Número máximo de iteraciones del solver (lbfgs por defecto) para converger la optimización MLE.
<code>LogisticRegression.class_weight</code>	"balanced"	Repondera la función de pérdida con pesos $\frac{n_samples}{(n_classes \times n_c)}$, inflando el peso de la clase minoritaria (convertidos, ~1-5%).

Hiperparámetro	Valor Asignado	Definición Técnica
<code>LogisticRegression.random_state</code>	42	Semilla del generador aleatorio interno (relevante para solvers estocásticos/desempates).
<code>LogisticRegression.penalty</code> / C (implícitos)	12 / 1.0 (defaults)	Regularización ridge sobre los coeficientes con fuerza inversa C.
<code>OneHotEncoder.handle_unknown</code>	"ignore"	Ante categorías no vistas en fit, emite vector de ceros en lugar de lanzar <code>ValueError</code> .

Hiperparámetro	Valor Asignado	Definición Técnica
StandardScaler	defaults (<u>with_mean=True,</u> <u>with_std=True</u>)	Estandariza cada numérica a media 0 y varianza 1 con estadísticos aprendidos en fit (dentro del Pipeline → sin fuga hacia test).
ColumnTransformer (passthrough de flags)	"passthrough" en <u>OFFER_FLAG_COLS</u>	Los 8 flags de acción entran al modelo sin transformación.
EPSILON_SCHEDULE	<u>np.linspace(1.0, 0.05, 10)</u>	Tasa de exploración por batch: fracción de clientes que recibe acción aleatoria uniforme en lugar de la greedy. Decae linealmente 1,00 → 0,05.

Hiperparámetro	Valor Asignado	Definición Técnica
RNG (semilla del bandit)	<code>np.random.default_rng(42)</code>	Generador PCG64 que gobierna toda la aleatoriedad de exploración (qué clientes exploran y qué acción reciben).
Tamaño de batch / nº de batches	<code>50.000 × 10</code>	Volumen de interacción por ciclo de actualización de política.
Régimen de reentrenamiento	Full-history refit por batch <code>((pd.concat(training_log)))</code>	En cada batch se reajusta el pipeline desde cero sobre TODOS los datos acumulados.

Hiperparámetros del modelo estadístico auxiliar (Celda 21, `sm.Logit`): sin configuración explícita más allá de `(disp=False)` (silencia el log de optimización); estimación MLE estándar con covarianza no robusta, adecuada para datos i.i.d. de un experimento aleatorizado.

3.3 Estrategia de Validación

El esquema **no es K-Fold ni Stratified K-Fold**, y esa ausencia es metodológicamente

correcta y merece explicación: en aprendizaje por interacción (bandits), la validación cruzada clásica no aplica directamente porque los datos de entrenamiento son **generados por la propia política** (on-policy) y el objetivo no es el error predictivo del clasificador, sino el **desempeño de la política de decisión en el entorno**. El notebook implementa en su lugar una validación de tres anillos, del más débil al más fuerte:

1. **Validación secuencial durante el entrenamiento (curva de aprendizaje, Celda 29).**
Cada batch actúa como evaluación *online* de la política vigente (contaminada por la fracción ϵ de exploración). La progresión 0,95% $\rightarrow$ 5,40% de conversión es evidencia de aprendizaje, análoga a un *forward-chaining* temporal.
2. **Holdout interno (Celda 30).** `test_leads_optimized`: 50.000 leads nunca usados en entrenamiento, evaluados en explotación pura contra los dos baselines **sobre la misma población** (comparación pareada que elimina varianza entre muestras). Verifica generalización a clientes no vistos.
3. **Validación oficial ciega (Celdas 32-34).** `final_test_validation_base.parquet`: base externa provista por el caso (no generada por el candidato), con las tres estrategias evaluadas nuevamente sobre población idéntica. Es el equivalente a un *held-out test set* de competencia, y sus números son los reportables.

La evaluación final es inherentemente estocástica (una realización de `sim()`); la consistencia de resultados entre el test interno (\$88k) y el oficial (\$134k) — ambos muy por encima de los baselines — mitiga el riesgo de que el lift sea ruido. Como mejora, se podrían repetir múltiples corridas de `sim()` para intervalos de confianza, si el contrato del caso lo permitiera.

3.4 Métricas de Evaluación — Desglose

El caso no usa métricas de clasificación clásicas (F1, AUC-ROC, Log-Loss) como criterio final, sino **métricas de negocio de la política**, lo cual es la elección correcta: el clasificador es un medio; la decisión económica es el fin. Métricas empleadas:

Métrica	Definición	Valor final (set oficial)	Interpretación
Conversion Rate	<code>converted.mean()</code> : fracción de clientes que aceptan la oferta.	Bandit 5,7% vs. Low-Cost 2,3% vs. Random 0,9%	Eficacia comercial de la primera oferta. El bandit convierte 2,5× más que la mejor táctica fija.
Total Margin	<code>total_margin.sum()</code> : margen realizado agregado (0 para no convertidos).	\$133.848 vs. \$28.644 vs. \$18.829	KPI primario (mandato de la CMO). Captura simultáneamente volumen y calidad de conversión.
Margin / Convert	Margen medio condicional a conversión.	\$46,75 vs. \$25,06 vs. \$40,06	Revela el mecanismo del lift: el bandit no solo convierte más, sino que convierte en ofertas de mayor valor que la low-cost (+87% de margen por venta).
Margin / Customer	<code>total_margin.mean()</code> : valor esperado por lead contactado.	\$2,68 vs. \$0,57 vs. \$0,38	Métrica de eficiencia de campaña, directamente escalable a P&L (por cada 1M de leads: \$2,68M vs. \$0,57M).
Lift (× y \$)	Ratio y diferencia de margen total vs. baselines.	7,11× / +\$115.020 vs. Random; 4,67× / +\$105.205 vs. Low-Cost	Cuantificación ejecutiva del valor incremental del modelo.
Cobertura del espacio de acciones	Nº de bundles distintos usados / 96.	6 (6,2%)	Diagnóstico de personalización (ver §4.3: la concentración observada matiza el título de "prueba de personalización").

Métricas estadísticas auxiliares (del modelo de elasticidad, Celda 21): Pseudo R^2 de McFadden (0,073 — bajo en absoluto, esperable en conversión binaria con base $<1\%$), test LLR ($p \approx 1,9e-64$: el modelo aporta información muy significativa sobre el nulo), p-values de Wald por coeficiente y odds ratios ($\exp(\beta)$). Estas fundamentan inferencia, no selección de política.

4. Celdas de Validación Final y Explicabilidad (31-37)

Celda 31 — [MARKDOWN | Sección "Validate your final result and report final KPIs"]

Separador de sección.

Celda 32 — [CÓDIGO | Preparación de datos — carga de la base oficial y baselines]

Código fuente:

```
python

#Do not modify this cell
import pandas as pd
final_test_validation_base = pd.read_parquet("data_and_supporting_code/final_test_validation_base.parquet")
display(final_test_validation_base.head())

random_offer_test = random_offer_fn(final_test_validation_base)
random_result_test = sim(random_offer_test)
low_cost_offer_test = fixed_offer(final_test_validation_base)
low_cost_result_test = sim(low_cost_offer_test)
```

Explicación técnica: Celda inmutable. `pd.read_parquet` carga

`final_test_validation_base` (50.000 leads; Parquet: formato columnar binario eficiente, preserva tipos). Ejecuta las dos estrategias baseline sobre esa base (`random_offer_fn` + `sim`, `fixed_offer` + `sim`), produciendo `random_result_test` y `low_cost_result_test`.

Impacto en el pipeline: Congela los benchmarks oficiales sobre la población de evaluación definitiva.

Celda 33 — [CÓDIGO | Modelado — inferencia oficial]

Código fuente:

```
python

#Write your code here, do not change the name of the offer test dataframe

optimized_offer_test = bandit_function(final_test_validation_base)
```

Explicación técnica: Única línea del candidato en la fase oficial: aplica la política final en explotación pura sobre la base oficial, respetando el nombre contractual del DataFrame. Ninguna llamada a `sim()` ocurre aquí — la asignación es puramente model-driven.

Impacto en el pipeline: Genera el artefacto que la celda inmutable siguiente someterá a simulación.

Celda 34 — [CÓDIGO | Evaluación — comparación de estrategias oficial]

Código fuente:

```
python
```

```

#Do not modify this cell
optimized_result_test = sim(optimized_offer_test)

# — Benchmark comparison —————
rows = []
for name, df in [
    ('Random Offer (baseline)', random_result_test),
    ('Low-Cost Offer (baseline)', low_cost_result_test),
    ('Contextual Bandit (ours)', optimized_result_test),
]:
    conv_df = df[df['converted']]
    rows.append({
        'Strategy': name,
        'Customers': f"{len(df):,}",
        'Converted': f"{df['converted'].sum():,}",
        'Conv. Rate': f"{df['converted'].mean():.1%}",
        'Total Margin': f"${df['total_margin'].sum():.0f}",
        'Margin / Convert': f"${conv_df['total_margin'].mean():.2f}" if len(c
        'Margin / Customer': f"${df['total_margin'].mean():.2f}",
    })
print("=== Strategy Comparison ===")
display(pd.DataFrame(rows))

# — Offer-mix comparison —————
offer_feature_cols = [c for c in optimized_offer_test.columns
                       if c.startswith('offer_') and c != 'offer_description']
mix = pd.DataFrame({
    'Optimized': optimized_offer_test[offer_feature_cols].mean(),
    'Low-Cost': low_cost_offer_test[offer_feature_cols].mean(),
    'Random': random_offer_test[offer_feature_cols].mean(),
})
fig, ax = plt.subplots(figsize=(12, 4))
mix.T.plot(kind='bar', ax=ax, width=0.7)
ax.set_title('Offer Component Incidence: Strategy Comparison')
ax.set_ylabel('Fraction of Customers Offered (0-1)')
ax.set_xticklabels(['Bandit', 'Low-Cost', 'Random'], rotation=0)
ax.legend(
    [c.replace('offer_', '').replace('_', ' ').title() for c in offer_feature_c
    loc='upper right', fontsize=8, ncol=2,
)
plt.tight_layout()
plt.show()

# — Lift summary —————

```

```
opt_m = optimized_result_test['total_margin'].sum()
lc_m = low_cost_result_test['total_margin'].sum()
rnd_m = random_result_test['total_margin'].sum()
print(f"Lift vs Random Offer: {opt_m/rnd_m:.2f}× (+${opt_m - rnd_m:,.0f})")
print(f"Lift vs Low-Cost Offer: {opt_m/lc_m:.2f}× (+${opt_m - lc_m:,.0f})")
```

Explicación técnica: Celda inmutable (con visualizaciones adjuntas).

`sim(optimized_offer_test)` produce el resultado oficial. Luego:

- Construye la tabla comparativa iterando las tres estrategias: clientes, convertidos, tasa, margen total, margen por convertido y por cliente (formateo con f-strings; guarda de división por cero con el condicional `if len(conv_df)`).
- **Comparación de mezcla de oferta:** `mean()` de cada flag por estrategia = incidencia de cada componente (fracción de clientes a los que se ofrece), graficada como barras agrupadas — visualiza *qué* decidió el bandit frente a los baselines.
- Resumen de lift en × y \$.

Resultados (transcritos en §0): conversión 5,7%, margen \$133.848, lift 7,11×/4,67×.

Impacto en el pipeline: Es el veredicto cuantitativo final del caso. Las Celdas 35-37 lo explican y comunican.

Celda 35 — [CÓDIGO | Explicabilidad — importancia de features (perspectiva marketing)]

Código fuente:

```
python
```

```

# =====
# PART 1: Feature Importance (Marketing Perspective)
# =====

from matplotlib.patches import Patch

sns.set_style("whitegrid")
PALETTE = "viridis"

fitted_preprocessor = bandit_pipeline.named_steps["preprocessor"]
fitted_classifier = bandit_pipeline.named_steps["classifier"]

feature_names = fitted_preprocessor.get_feature_names_out()
coefficients = fitted_classifier.coef_[0]
assert len(feature_names) == len(coefficients), "Feature name / coefficient len

def clean_feature_name(name: str) -> str:

    name = name.split("__", 1)[-1]
    name = name.replace("offer_", "")
    return name.replace("_", " ").title()

feature_importance = pd.DataFrame({
    "feature": [clean_feature_name(f) for f in feature_names],
    "coefficient": coefficients,
})
feature_importance["abs_coefficient"] = feature_importance["coefficient"].abs()
feature_importance["direction"] = np.where(
    feature_importance["coefficient"] >= 0, "Increases Conversion Odds", "Decreases Conversion Odds"
)
feature_importance = feature_importance.sort_values("abs_coefficient", ascending=False)

TOP_N = 20
top_features = feature_importance.head(TOP_N).sort_values("abs_coefficient")
bar_colors = top_features["direction"].map({
    "Increases Conversion Odds": sns.color_palette(PALETTE, 2)[1],
    "Decreases Conversion Odds": sns.color_palette(PALETTE, 2)[0],
})

plt.figure(figsize=(10, 8))
plt.barh(top_features["feature"], top_features["coefficient"], color=bar_colors)

```

```
plt.axvline(0, color="grey", linewidth=0.8)
plt.title(f"Top {TOP_N} Features Driving the Bandit's Offer Decisions\n(Logistic Regression)")
plt.xlabel("Coefficient (Impact on Log-Odds of Conversion)")
plt.ylabel("")
plt.legend(
    handles=[
        Patch(facecolor=sns.color_palette(PALETTE, 2)[1], label="Increases Conversion"),
        Patch(facecolor=sns.color_palette(PALETTE, 2)[0], label="Decreases Conversion"),
    ],
    loc="lower right",
)
plt.tight_layout()
plt.show()

print("\nTop 5 features driving offer decisions:")
print(feature_importance.head(5)[["feature", "coefficient", "direction"]].to_string())
```

Explicación técnica:

- Extrae del `Pipeline` entrenado el preprocesador y el clasificador ajustados (`named_steps`), obtiene los nombres de features post-transformación con `get_feature_names_out()` (incluye las dummies one-hot expandidas) y los coeficientes `coef_[0]`. Un `assert` verifica el alineamiento longitud-a-longitud.
- `clean_feature_name` limpia los prefijos técnicos de sklearn (`categorical__`, `offer__`) para legibilidad de negocio.
- Ordena por `|coeficiente|` (magnitud de impacto en log-odds — comparable entre features gracias a la estandarización previa), clasifica dirección (aumenta/disminuye odds) y grafica el top-20 como barras horizontales bicolor con `matplotlib.patches.Patch` para la leyenda manual.

Top-5 drivers observados:

Feature	Coef.	Dirección
Segmento A	+2,83	↑ conversión
Voice Line (oferta)	-2,61	↓ conversión
Movilidad Home-Work-Third Site	-2,17	↓ conversión
Movilidad Home-Mostly One Cell	+2,14	↑ conversión
iPhone Mid End	-1,57	↓ conversión

Triangulación metodológica (hallazgo de auditoría): el bandit redescubre de forma independiente el efecto destructivo de la línea de voz que el logit de elasticidad ya había estimado ($-1,31, p<0,001$) sobre datos distintos, y confirma las hipótesis del EDA (afluencia y permanencia en hogar como drivers positivos). La convergencia de tres análisis independientes eleva sustancialmente la confianza en las conclusiones.

Impacto en el pipeline: Traduce el modelo a lenguaje accionable de marketing y sirve como control de plausibilidad del aprendizaje.

Celda 36 — [CÓDIGO | Evaluación — impacto financiero (perspectiva ejecutiva)]

Código fuente:

```
python
```

```

# =====
# PART 2: Financial Impact Comparison (Executive Perspective)
# =====

scenarios = {
    "Random Offer": random_result_test,
    "Low-Cost Offer": low_cost_result_test,
    "Contextual Bandit": optimized_result_test,
}

# Total expected margin: sum of total_margin among converted customers only
# (equal to summing over all rows, since non-converted rows carry margin 0 -
# filtering explicitly keeps the calculation self-documenting)
total_margin_by_scenario = {
    name: df.loc[df["converted"], "total_margin"].sum()
    for name, df in scenarios.items()
}

margin_summary = pd.DataFrame({
    "Scenario": list(total_margin_by_scenario.keys()),
    "Total Expected Margin ($)": list(total_margin_by_scenario.values()),
})

print("\n=== Total Expected Margin by Scenario (final_test_validation_base) ===")
print(margin_summary.round(2))

bandit_margin = total_margin_by_scenario["Contextual Bandit"]
random_margin = total_margin_by_scenario["Random Offer"]
low_cost_margin = total_margin_by_scenario["Low-Cost Offer"]

lift_vs_random_abs = bandit_margin - random_margin
lift_vs_random_pct = (lift_vs_random_abs / random_margin) * 100
lift_vs_low_cost_abs = bandit_margin - low_cost_margin
lift_vs_low_cost_pct = (lift_vs_low_cost_abs / low_cost_margin) * 100

print("\n=== Financial Lift: Contextual Bandit vs. Baselines ===")
print(f"vs. Random Offer:    ${lift_vs_random_abs:,.2f} ({lift_vs_random_pct:+.1f}%)")
print(f"vs. Low-Cost Offer:  ${lift_vs_low_cost_abs:,.2f} ({lift_vs_low_cost_pct:+.1f}%)")

colors = sns.color_palette(PALETTE, 3)
fig, ax = plt.subplots(figsize=(10, 7))
bars = ax.bar(margin_summary["Scenario"], margin_summary["Total Expected Margin"],
              color=colors)

for bar in bars:
    height = bar.get_height()
    ax.annotate(

```

```

        f"${height:,.0f}",
        xy=(bar.get_x() + bar.get_width() / 2, height),
        xytext=(0, 6), textcoords="offset points",
        ha="center", va="bottom", fontsize=11, fontweight="bold",
    )

bandit_bar = bars[2]
bandit_top = bandit_bar.get_height()
ax.annotate(
    f"+{lift_vs_random_pct:.0f}% (+${lift_vs_random_abs:,.0f}) vs. Random\n"
    f"+{lift_vs_low_cost_pct:.0f}% (+${lift_vs_low_cost_abs:,.0f}) vs. Low-Cost",
    xy=(bandit_bar.get_x() + bandit_bar.get_width() / 2, bandit_top),
    xytext=(0, 34), textcoords="offset points",
    ha="center", va="bottom", fontsize=10, fontweight="bold", color="darkgreen",
    bbox=dict(boxstyle="round,pad=0.3", fc="honeydew", ec="darkgreen", lw=0.8),
)

ax.set_title("Financial Impact: Total Expected Margin by Offer Strategy\n(Office)")
ax.set_xlabel("")
ax.set_ylabel("Total Expected Margin ($)")
ax.set_ylim(0, bandit_top * 1.3)
plt.tight_layout()
plt.show()

```

Explicación técnica: Reagrega el margen total por escenario filtrando explícitamente convertidos (`(df.loc[df["converted"], "total_margin"].sum())`); el comentario aclara que equivale a sumar todo, pues los no convertidos aportan 0 — el filtro es auto-documentación). Calcula lifts absolutos y porcentuales, y produce un gráfico de barras ejecutivo con anotaciones de valor sobre cada barra (`ax.annotate` con offsets en puntos) y un callout destacado sobre la barra del bandit (+610,9% vs. Random; +367,3% vs. Low-Cost) con caja estilizada (`bbox`). **Impacto en el pipeline:** Pieza de comunicación C-level; no altera datos.

Celda 37 — [CÓDIGO | Evaluación — distribución de ofertas sugeridas]

Código fuente:

```
python
```

```

# =====
# PART 3: Suggested Offer Distribution (Personalization Proof)
# =====

offer_results = optimized_result_test.loc[:, ~optimized_result_test.columns.duplicated()]

offer_distribution = (
    offer_results["offer_description"]
    .value_counts(normalize=True)
    .mul(100)
    .rename("pct_of_leads")
    .reset_index()
    .rename(columns={"index": "offer_description"})
    .sort_values("pct_of_leads", ascending=False)
)

print("\n=== Suggested Offer Distribution (Contextual Bandit, Official Test Set)")
print(offer_distribution.round(2).to_string(index=False))

plot_data = offer_distribution.sort_values("pct_of_leads")
fig_height = max(5, 0.4 * len(plot_data))
plt.figure(figsize=(11, fig_height))
bars = plt.barh(
    plot_data["offer_description"], plot_data["pct_of_leads"],
    color=sns.color_palette(PALETTE, len(plot_data)),
)

for bar, pct in zip(bars, plot_data["pct_of_leads"]):
    plt.text(
        bar.get_width() + 0.3, bar.get_y() + bar.get_height() / 2,
        f"{pct:.1f}%", va="center", ha="left", fontsize=9,
    )

plt.title("Suggested Offer Distribution – Contextual Bandit Personalization\n(Official Test Set)")
plt.xlabel("Share of Customer Base (%)")
plt.ylabel("")
plt.xlim(0, plot_data["pct_of_leads"].max() * 1.15)
plt.tight_layout()
plt.show()

n_unique_offers_used = offer_results["offer_description"].nunique()
n_total_action_space = len(ACTION_TABLE)

# =====
# Consolidated Summary
# =====

```

```

print("\n=== Executive Summary ===")
print(
    f"Financial Lift: the Contextual Bandit generated ${bandit_margin:,.2f} in
    f"expected margin on the official test set – a gain of ${lift_vs_random_abs
    f"({lift_vs_random_pct:+.1f}%) over Random and ${lift_vs_low_cost_abs:,.2f}
    f"({lift_vs_low_cost_pct:+.1f}%) over the fixed Low-Cost strategy."
)
print(
    f"Personalization: the policy actively used {n_unique_offers_used} distinct
    f"combinations out of {n_total_action_space} valid actions "
    f"({n_unique_offers_used / n_total_action_space:.1%} of the space), confirm
    "tailors offers to customer context rather than defaulting to one bundle."
)

```

Explicación técnica: Sobre el resultado oficial (deduplicando columnas de nuevo por robustez), computa `value_counts(normalize=True)` de `offer_description` → distribución porcentual de bundles recomendados, tabulada y graficada en barras horizontales con etiquetas de porcentaje. Cierra con un resumen ejecutivo impreso: margen final, lifts y cobertura del espacio de acciones.

Distribución observada:

Bundle recomendado	% de leads
High Speed + Wall-to-Wall WiFi	77,67%
High Speed + Premium TV + WiFi	9,32%
High Speed + Premium TV + Disney + WiFi	6,89%
High Speed + Disney + WiFi	3,26%
High Speed + Premium TV + Disney + Paramount + WiFi	2,64%
High Speed + Premium TV + Disney + Paramount + Voz + WiFi	0,21%

Lectura estratégica del auditor: la política converge a una lógica económica nítida: (a) **siempre High Speed** — su margen (\$31,61) domina tanto que compensa la menor conversión por precio; (b) **siempre WiFi** — a \$4 apenas deprime la conversión y suma margen casi gratis; (c) **la voz se elimina** casi por completo (0,21%) — consistente con su coeficiente fuertemente negativo; (d) los add-ons de contenido (Premium TV, Disney, Paramount) se dosifican en ~22% de la base según el perfil del cliente.

Matiz crítico (hallazgo de auditoría): el resumen final afirma que usar 6 de 96 bundles "confirma personalización". La afirmación es defendible pero optimista: con 77,7% de la base en un único bundle, la personalización efectiva opera sobre ~22% de los clientes. Es plausible que este sea el óptimo real del entorno (una oferta dominante + ajustes marginales), pero también podría reflejar la limitación aditiva del modelo lineal (§3.1). Recomendación: contrastar con un reward model con interacciones antes de concluir que el techo de personalización se alcanzó.

Impacto en el pipeline: Cierre del notebook; evidencia final de comportamiento de la política.

5. Dictamen de Auditoría

5.1 Cumplimiento del contrato del caso

Requisito	Cumplimiento	Evidencia
Solo 10 batches de entrenamiento (500k leads)	✓	El bucle consume exactamente <code>train_leads_optimized_1..10</code> ; una llamada a <code>sim()</code> por batch.
No ingeniería inversa de <code>sim()</code>	✓	<code>sim()</code> se trata como oráculo de recompensa; la selección de acciones nunca lo consulta (docstring explícito en <code>select_best_actions</code>).
No fuerza bruta contra el entorno	✓	La evaluación exhaustiva de 96 acciones ocurre solo contra el modelo interno (scoring gratuito), no contra <code>sim()</code> .
Política basada en aprendizaje con exploración/explotación	✓	ϵ -greedy con annealing 1,0→0,05; cold start de exploración total.
Generalización (no lookup table)	✓	Modelo paramétrico evaluado en dos poblaciones no vistas con lift consistente.
Columnas originales sin transformar + flags contractuales	✓	El ensamblado copia el batch original y solo añade flags; restricciones de oferta garantizadas por construcción con asserts.

5.2 Fortalezas técnicas destacadas

1. **Alineación objetivo-negocio:** optimiza margen esperado, no conversión — decisión que explica que el bandit gane con ofertas *más caras* que la low-cost.
2. **Restricciones por construcción** con verificación formal (asserts en espacio de acciones y rango de la matriz de diseño).
3. **Triangulación:** EDA → logit inferencial → coeficientes del bandit convergen en las mismas conclusiones sobre datos distintos.
4. **Reproducibilidad total** (semillas explícitas, generador inyectado) y **robustez de inferencia** (`handle_unknown="ignore"`), deduplicación defensiva de columnas, guardas de división por cero).
5. **Honestidad analítica:** el propio código advierte límites (confusión precio/mix entre experimentos, poder estadístico limitado del efecto precio).

5.3 Oportunidades de mejora (no bloqueantes)

1. **Reward model con interacciones contexto×acción** (términos cruzados explícitos o GBM calibrado) para elevar el techo de personalización más allá del 22% de la base.
2. **Exploración dirigida** (Thompson Sampling con posterior bayesiano del logit, o LinUCB) en lugar de ϵ -greedy uniforme: mejor regret con el mismo presupuesto.
3. **Intervalos de confianza del lift** vía múltiples realizaciones de `sim()` sobre el set oficial (si el contrato lo permitiera) o bootstrap sobre conversiones.
4. **Calibración de probabilidades** (Platt/isotónica) si las $P(\text{conversión})$ fueran a usarse en valor absoluto (p. ej., para pricing dinámico) y no solo para ranking.
5. En datos reales: **split aleatorizado con verificación de balance** de covariables en lugar del corte posicional de la Celda 15, y monitoreo de *drift* entre campañas.

Fin del documento de auditoría.